

LECTURE II - COMPLEXITY

Book coverage: 1.5, 1.6.1, 1.6.4

WE CAN PROVE IT.WE CAN'T PRICE IT.

SETTLED

mean is correct for every input — initialize, maintain, terminate.

OPEN

5 seconds on 1,000 points. How long on 100,000 — on any machine?

Pillar 2 starts here. Correctness was a proof about values. Efficiency is a proof about counts.

COUNTING WITHOUT A STOPWATCH

A cost model that describes the algorithm instead of the computer.

SUPPOSE WE JUST RACE THEM

DIFFERENT MACHINES

My lab box may simply be faster than your laptop. The winner tells you about the hardware.

DIFFERENT HARDWARE FITS

Mine may be fast on a GPU and slow without one. Same algorithm, different verdict.

DIFFERENT INPUTS

Some inputs are easy and some are hard. One benchmark is one data point.

DIFFERENT SIZES

Mine can lose at $n = 100$ and win at $n = 1,000$. Overhead dominates early.

Every flaw has the same shape: the stopwatch measures a run, and we want a property of the procedure.

Not "how many seconds?"
but "how does it grow?"

GIVEN: The pseudocode of an algorithm, and an input size n .

COMPUTE: $T(n)$, the number of steps it takes, up to constants.

This is a question about the algorithm — not about the implementation underneath it.

ONE ASSUMPTION, TWO COLUMNS

ASSUMPTION

Each line takes some constant time c_i per execution — a number that depends on the machine, never on n .

So the whole running time is a sum of two things you can read off the page:

TIME PER EXECUTION

A machine fact. Call it c_i and move on.

NUMBER OF EXECUTIONS

An algorithm fact, in terms of n . This is the part we work for.

$T(n) = \sum (\text{time per execution}) \times (\text{number of executions}), \text{ over every line.}$

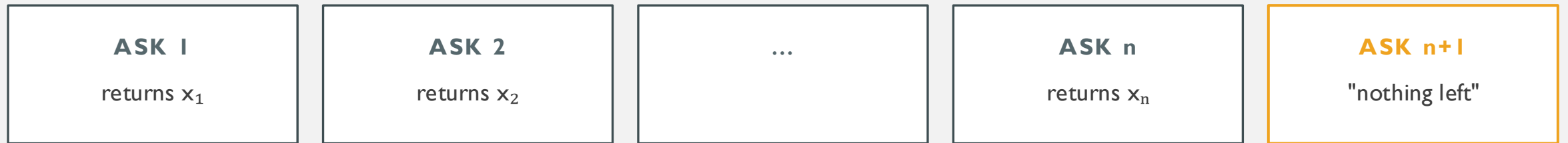
THE COST OF mean

LINE	TIME PER EXEC.	NUMBER OF EXECS.
total = 0	c ₁	1
n = len(data)	c ₂	1
for x in data:	c ₃	n + 1
total += x	c ₄	n
return total / n	c ₅	1

Every line is constant time per execution. Only the last column knows what n is.

WHY THE LOOP HEADER RUNS $n + 1$ TIMES

for x in data: does invisible work: ask the list for its next element, and bind it to x.



The loop body runs n times. The loop header runs $n + 1$ — the extra ask is how the loop learns to stop.

Worth getting right once, and then worth forgetting: by the end of this lecture that $+1$ will be invisible.

MULTIPLY, THEN SUM

T(n) IS A LINE

$$\begin{aligned}T(n) &= c_1 + c_2 + c_3(n + 1) + c_4 \cdot n + c_5 \\&= (c_3 + c_4) \cdot n + (c_1 + c_2 + c_3 + c_5) \\&= \mathbf{c \cdot n + c'}\end{aligned}$$

Two constants. We never measured either one, and we are never going to. What we learned is the **shape**:

T is linear in n.

c and c' belong to the machine.

The word "linear" belongs to the algorithm.

Buy a machine twice as fast and c halves. Nothing you buy will make $T(n)$ stop being a line.

SCALABILITY AND BIG-THETA

What survives when you throw the constants away and why that is the useful part.

WHAT HAPPENS IF I DOUBLE THE DATA?

$$\frac{T(2n)}{T(n)} = \frac{2cn + c'}{cn + c'}$$

SPLIT IT

$$\frac{2cn}{cn + c'} + \frac{c'}{cn + c'}$$

as n grows.

$$T(2n) \approx 2 \cdot T(n) \quad \text{for large } n.$$

The constants cancelled out of the answer. That is not luck — it is the reason we bothered to keep them symbolic.

SCALABILITY

DEFINITION

How the run time of an algorithm grows as the input grows.

AT SMALL n

Everything is fast. A terrible algorithm and a great one are indistinguishable on 100 rows.

AT LARGE n

The scaling behaviour is the only thing left. It decides whether the job finishes at all.

mean scales linearly: double the input, roughly double the time. Soon we will meet algorithms where doubling costs far more.

Θ , INFORMALLY

Drop every constant factor. Drop every lower-order term. Keep what is left.

YOU DERIVED	YOU WRITE	YOU SAY
$T(n) = c \cdot n + c'$	$T(n) = \Theta(n)$	linear time
$T(n) = a \cdot n^2 + b \cdot n + c$	$T(n) = \Theta(n^2)$	quadratic time
$T(n) = c_1 + c_2 + c_3$	$T(n) = \Theta(1)$	constant time

Θ has a precise definition with constants c_1, c_2 and a threshold n_0 — that is Lecture 3. Today we use it as a filter.

READ THE FINE PRINT

Θ IS DELIBERATELY LOSSY

KEPT

The growth rate. Whether doubling n doubles the time, quadruples it, or worse.

DISCARDED

Constant factors and lower-order terms. An algorithm twice as fast looks identical to Θ .

Both facts are useful. Θ answers "will this finish?" — it does not answer "which of these two should I ship?"

YOU DO NOT NEED THE WHOLE TABLE

01 Check every line is constant time.

No hidden loops, no calls that grow with n . If this fails, you have more work to do first.

02 Find the line executed most often.

As a function of n . It is almost always the innermost loop.

03 Report that count, filtered by Θ .

Everything else is a lower-order term, and Θ was built to erase those.

WHERE THIS GOES NEXT

ONE LOOP OVER N ELEMENTS GIVES $\Theta(N)$

WHAT DOES A LOOP INSIDE A LOOP GIVE?

The obvious answer is n^2 . The obvious answer is wrong often enough to be worth a module.

NESTED LOOPS

THE MEAN OF AN $n \times n$ ARRAY

```
def array_mean(arr):  
    total = 0  
    n = len(arr)  
    for i in range(n):  
        row_total = 0  
        for j in range(n):  
            row_total += arr[i, j]  
        total += row_total  
    return total / n**2
```

We proved this correct last lecture — two invariants, inner handed to outer.

Every line is still constant time per execution, so the whole job is the number of executions.

Note n is the number of rows, so the array holds n^2 numbers. Input size and n are not the same thing here.

THE THIRD COLUMN, AND ONLY THE THIRD

LINE	NUMBER OF EXECS.
total = 0	1
n = len(arr)	1
for i in range(n):	n + 1
row_total = 0	n
for j in range(n):	n(n + 1)
row_total += arr[i, j]	n ²
total += row_total	n
return total / n**2	1

The inner header runs $n + 1$ times per outer pass, and the outer pass happens n times — hence $n(n + 1)$.

THE PAYOUT

$$n(n + 1) = n^2 + n = \Theta(n^2)$$

array_mean is quadratic. Double the side length and the work goes up fourfold.

The inner body runs n^2 times and the inner header runs $n^2 + n$ times. **Θ cannot tell them apart** — so picking the wrong one of the two costs you nothing.

TWO LOOPS DOES NOT MEAN n^2

```
def nested_algorithm(data):  
    for x in data:  
        for y in range(4):  
            print(x * y)
```

range(4) does not mention n . The inner loop's trip count is fixed.

Most-executed line: the inner header, at $(4 + 1) \cdot n = 5n$ times.

$T(n) = \Theta(n)$. Linear.

Depth of nesting is not the thing you count. Executions are the thing you count.

WHEN THE INNER RANGE MOVES

```
def symmetric_array_mean(arr):  
    total = 0  
    n = len(arr)  
    for i in range(n):  
        total += arr[i, i]  
        for j in range(i+1, n):  
            total += 2 * arr[i, j]  
    return total / n**2
```

A symmetric array mirrors over its diagonal, so half its entries are duplicates.

This version touches only the diagonal and the entries to its right, counting each off-diagonal entry twice.

It is clearly doing less work than `array_mean`. The question is whether Θ notices.

HOW MANY TIMES DOES THE INNER BODY RUN?

IT DEPENDS ON WHICH PASS YOU ARE ON

Take $n = 4$. On the α th pass of the outer loop, the inner range is $\text{range}(\alpha, 4)$.

$\alpha = 1$ $\text{range}(1, 4)$ 3	$\alpha = 2$ $\text{range}(2, 4)$ 2	$\alpha = 3$ $\text{range}(3, 4)$ 1	$\alpha = 4$ $\text{range}(4, 4)$ 0
--	--	--	--

In general: on the α th pass, the inner body runs **$n - \alpha$** times.

Note: α is the mathematical iteration counter from Lecture 1 — the first pass is $\alpha = 1$. i is whatever Python happens to hold.

SUM OVER EVERY PASS

AN ARITHMETIC SERIES

$$(n - 1) + (n - 2) + \dots + 2 + 1 + 0$$

= the sum of the first $n - 1$ natural numbers

$$= n(n - 1) / 2$$

$S_k = k(k + 1)/2$ is worth memorising. Careful with the substitution: here $k = n - 1$, not n .

$$n(n - 1) / 2 = \Theta(n^2)$$

Same complexity as `array_mean`.

And yet `symmetric_array_mean` really is about twice as fast. That factor of two lives entirely in the constant Θ throws away. Time complexity is not the whole story about performance — it is the part that does not depend on your machine.

YOUR TURN

```
def triangle_sum(arr):  
    n = len(arr)  
    total = 0  
    for i in range(n):  
        for j in range(i):  
            total += arr[i, j]  
    for k in range(n):  
        total += arr[k, k]  
    return total
```

Pause here. Write it by hand first.

Give $T(n)$ in Θ -notation.

WATCH FOR

Two loop nests, one after the other — not one inside the other. Count each, then decide what their sum is worth.

MY TURN

LINE	NUMBER OF EXECS.
n = len(arr)	1
total = 0	1
for i in range(n):	n + 1
for j in range(i):	n(n+1)/2
total += arr[i, j]	n(n-1)/2
for k in range(n):	n+1
total += arr[k, k]	n
return total	1

For each i, the header runs i+1 times. Adding that up across all n iterations (1+2+...+n) gives n(n+1)/2.

Same idea for the body: it runs i times per iteration, and 0+1+...+(n-1) sums to n(n-1)/2.

THE PAYOUT

$$(n+1) + n(n+1)/2 + n(n-1)/2 + (n+1) + c = \Theta(n^2)$$

triangle_sum is quadratic.

WHAT YOU CAN DO NOW

Build a time-per-execution / number-of-executions table for any loop.

Collapse it to $T(n)$, then filter it through Θ .

Skip the table: find the most-executed line and read the complexity off it.

Handle a nested loop whose inner range depends on the outer counter.

Say what Θ has thrown away, and when that matters.

NEXT — LECTURE 3, COMPLEXITY II. We have been using Θ on a promise. Time to define it: c_1, c_2, n_0 — plus O and Ω .